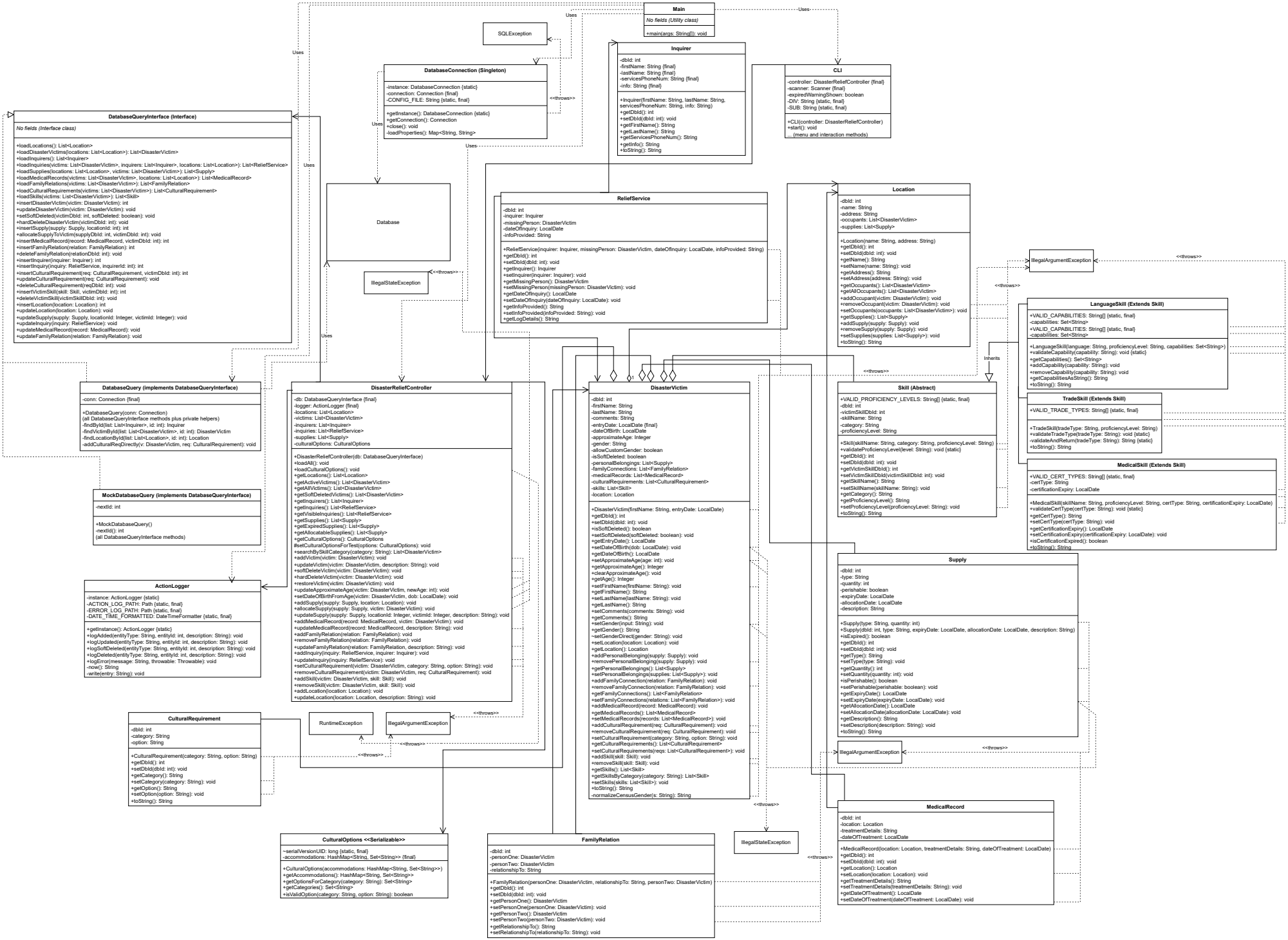
Main
No fields (Utility class)
+main(args: String[]): void
SQLException
Uses
Inquirer
-dbId: int
-firstName: String {final}
-lastName: String {final}
-servicesPhoneNum: String {final}
-info: String {final}
+Inquirer(firstName: String, lastName: String, servicesPhoneNum: String, info: String)
+getDbId(): int
+setDbId(dbId: int): void
+getFirstName(): String
+getLastName(): String
+getServicesPhoneNum(): String
+getInfo(): String
+toString(): String
CLI
-controller: DisasterReliefController {final}
-scanner: Scanner {final}
-expiredWarningShown: boolean
-DIV: String {static, final}
-SUB: String {static, final}
+CLI(controller: DisasterReliefController)
+start(): void
... (menu and interaction methods)
DatabaseConnection (Singleton)
-instance: DatabaseConnection {static}
-connection: Connection {final}
-CONFIG_FILE: String {static, final}
+getInstance(): DatabaseConnection {static}
+getConnection(): Connection
+close(): void
-loadProperties(): Map<String, String>
<<throws>>
DatabaseQueryInterface (Interface)
No fields (Interface class)
+loadLocations(): List<Location>
+loadDisasterVictims(locations: List<Location>): List<DisasterVictim>
+loadInquirers(): List<Inquirer>
+loadInquiries(victims: List<DisasterVictim>, inquirers: List<Inquirer>, locations: List<Location>): List<ReliefService>
+loadSupplies(locations: List<Location>, victims: List<DisasterVictim>): List<Supply>
+loadMedicalRecords(victims: List<DisasterVictim>, locations: List<Location>): List<MedicalRecord>
+loadFamilyRelations(victims: List<DisasterVictim>): List<FamilyRelation>
+loadCulturalRequirements(victims: List<DisasterVictim>): List<CulturalRequirement>
+loadSkills(victims: List<DisasterVictim>): List<Skill>
+insertDisasterVictim(victim: DisasterVictim): int
+updateDisasterVictim(victim: DisasterVictim): void
+setSoftDeleted(victimDbId: int, softDeleted: boolean): void
+hardDeleteDisasterVictim(victimDbId: int): void
+insertSupply(supply: Supply, locationId: int): int
+allocateSupplyToVictim(supplyDbId: int, victimDbId: int): void
+insertMedicalRecord(record: MedicalRecord, victimDbId: int): int
+insertFamilyRelation(relation: FamilyRelation): int
+deleteFamilyRelation(relationDbId: int): void
+insertInquirer(inquirer: Inquirer): int
+insertInquiry(inquiry: ReliefService, inquirerId: int): int
+insertCulturalRequirement(req: CulturalRequirement, victimDbId: int): int
+updateCulturalRequirement(req: CulturalRequirement): void
+deleteCulturalRequirement(reqDbId: int): void
+insertVictimSkill(skill: Skill, victimDbId: int): int
+deleteVictimSkill(victimSkillDbId: int): void
+insertLocation(location: Location): int
+updateLocation(location: Location): void
+updateSupply(supply: Supply, locationId: Integer, victimId: Integer): void
+updateInquiry(inquiry: ReliefService): void
+updateMedicalRecord(record: MedicalRecord): void
+updateFamilyRelation(relation: FamilyRelation): void
Database
IllegalStateException
ReliefService
-dbId: int
-inquirer: Inquirer
-missingPerson: DisasterVictim
-dateOfInquiry: LocalDate
-infoProvided: String
+ReliefService(inquirer: Inquirer, missingPerson: DisasterVictim, dateOfInquiry: LocalDate, infoProvided: String)
+getDbId(): int
+setDbId(dbId: int): void
+getInquirer(): Inquirer
+setInquirer(inquirer: Inquirer): void
+getMissingPerson(): DisasterVictim
+setMissingPerson(missingPerson: DisasterVictim): void
+getDateOfInquiry(): LocalDate
+setDateOfInquiry(dateOfInquiry: LocalDate): void
+getInfoProvided(): String
+setInfoProvided(infoProvided: String): void
+getLogDetails(): String
Location
-dbId: int
-name: String
-address: String
-occupants: List<DisasterVictim>
-supplies: List<Supply>
+Location(name: String, address: String)
+getDbId(): int
+setDbId(dbId: int): void
+getName(): String
+setName(name: String): void
+getAddress(): String
+setAddress(address: String): void
+getOccupants(): List<DisasterVictim>
+getAllOccupants(): List<DisasterVictim>
+addOccupant(victim: DisasterVictim): void
+removeOccupant(victim: DisasterVictim): void
+setOccupants(occupants: List<DisasterVictim>): void
+getSupplies(): List<Supply>
+addSupply(supply: Supply): void
+removeSupply(supply: Supply): void
+setSupplies(supplies: List<Supply>): void
+toString(): String
IllegalArgumentException
LanguageSkill (Extends Skill)
+VALID_CAPABILITIES: String[] {static, final}
-capabilities: Set<String>
+VALID_CAPABILITIES: String[] {static, final}
-capabilities: Set<String>
+LanguageSkill(language: String, proficiencyLevel: String, capabilities: Set<String>)
+validateCapability(capability: String): void {static}
+getCapabilities(): Set<String>
+addCapability(capability: String): void
+removeCapability(capability: String): void
+getCapabilitiesAsString(): String
+toString(): String
TradeSkill (Extends Skill)
+VALID_TRADE_TYPES: String[] {static, final}
+TradeSkill(tradeType: String, proficiencyLevel: String)
+validateTradeType(tradeType: String): void {static}
+validateAndReturn(tradeType: String): String {static}
+toString(): String
MedicalSkill (Extends Skill)
+VALID_CERT_TYPES: String[] {static, final}
-certType: String
-certificationExpiry: LocalDate
+MedicalSkill(skillName: String, proficiencyLevel: String, certType: String, certificationExpiry: LocalDate)
+validateCertType(certType: String): void {static}
+getCertType(): String
+setCertType(certType: String): void
+getCertificationExpiry(): LocalDate
+setCertificationExpiry(certificationExpiry: LocalDate): void
+isCertificationExpired(): boolean
+toString(): String
Inherits
DatabaseQuery (implements DatabaseQueryInterface)
-conn: Connection {final}
+DatabaseQuery(conn: Connection)
(all DatabaseQueryInterface methods plus private helpers)
-findById(list: List<Inquirer>, id: int): Inquirer
-findVictimById(list: List<DisasterVictim>, id: int): DisasterVictim
-findLocationById(list: List<Location>, id: int): Location
-addCulturalReqDirectly(v: DisasterVictim, req: CulturalRequirement): void
DisasterReliefController
-db: DatabaseQueryInterface {final}
-logger: ActionLogger {final}
-locations: List<Location>
-victims: List<DisasterVictim>
-inquirers: List<Inquirer>
-inquiries: List<ReliefService>
-supplies: List<Supply>
-culturalOptions: CulturalOptions
+DisasterReliefController(db: DatabaseQueryInterface)
+loadAll(): void
+loadCulturalOptions(): void
+getLocations(): List<Location>
+getActiveVictims(): List<DisasterVictim>
+getAllVictims(): List<DisasterVictim>
+getSoftDeletedVictims(): List<DisasterVictim>
+getInquirers(): List<Inquirer>
+getInquiries(): List<ReliefService>
+getVisibleInquiries(): List<ReliefService>
+getSupplies(): List<Supply>
+getExpiredSupplies(): List<Supply>
+getAllocatableSupplies(): List<Supply>
+getCulturalOptions(): CulturalOptions
#setCulturalOptionsForTest(options: CulturalOptions): void
+searchBySkillCategory(category: String): List<DisasterVictim>
+addVictim(victim: DisasterVictim): void
+updateVictim(victim: DisasterVictim, description: String): void
+softDeleteVictim(victim: DisasterVictim): void
+hardDeleteVictim(victim: DisasterVictim): void
+restoreVictim(victim: DisasterVictim): void
+updateApproximateAge(victim: DisasterVictim, newAge: int): void
+setDateOfBirthFromAge(victim: DisasterVictim, dob: LocalDate): void
+addSupply(supply: Supply, location: Location): void
+allocateSupply(supply: Supply, victim: DisasterVictim): void
+updateSupply(supply: Supply, locationId: Integer, victimId: Integer, description: String): void
+addMedicalRecord(record: MedicalRecord, victim: DisasterVictim): void
+updateMedicalRecord(record: MedicalRecord, description: String): void
+addFamilyRelation(relation: FamilyRelation): void
+removeFamilyRelation(relation: FamilyRelation): void
+updateFamilyRelation(relation: FamilyRelation, description: String): void
+addInquiry(inquiry: ReliefService, inquirer: Inquirer): void
+updateInquiry(inquiry: ReliefService): void
+setCulturalRequirement(victim: DisasterVictim, category: String, option: String): void
+removeCulturalRequirement(victim: DisasterVictim, req: CulturalRequirement): void
+addSkill(victim: DisasterVictim, skill: Skill): void
+removeSkill(victim: DisasterVictim, skill: Skill): void
+addLocation(location: Location): void
+updateLocation(location: Location, description: String): void
DisasterVictim
-dbId: int
-firstName: String
-lastName: String
-comments: String
-entryDate: LocalDate {final}
-dateOfBirth: LocalDate
-approximateAge: Integer
-gender: String
-allowCustomGender: boolean
-isSoftDeleted: boolean
-personalBelongings: List<Supply>
-familyConnections: List<FamilyRelation>
-medicalRecords: List<MedicalRecord>
-culturalRequirements: List<CulturalRequirement>
-skills: List<Skill>
-location: Location
+DisasterVictim(firstName: String, entryDate: LocalDate)
+getDbId(): int
+setDbId(dbId: int): void
+isSoftDeleted(): boolean
+setSoftDeleted(softDeleted: boolean): void
+getEntryDate(): LocalDate
+setDateOfBirth(dob: LocalDate): void
+getDateOfBirth(): LocalDate
+setApproximateAge(age: int): void
+getApproximateAge(): Integer
+clearApproximateAge(): void
+getAge(): Integer
+setFirstName(firstName: String): void
+getFirstName(): String
+setLastName(lastName: String): void
+getLastName(): String
+setComments(comments: String): void
+getComments(): String
+setGender(input: String): void
+getGender(): String
~setGenderDirect(gender: String): void
+setLocation(location: Location): void
+getLocation(): Location
+addPersonalBelonging(supply: Supply): void
+removePersonalBelonging(supply: Supply): void
+getPersonalBelongings(): List<Supply>
+setPersonalBelongings(supplies: List<Supply>): void
+addFamilyConnection(relation: FamilyRelation): void
+removeFamilyConnection(relation: FamilyRelation): void
+getFamilyConnections(): List<FamilyRelation>
+setFamilyConnections(relations: List<FamilyRelation>): void
+addMedicalRecord(record: MedicalRecord): void
+getMedicalRecords(): List<MedicalRecord>
+setMedicalRecords(records: List<MedicalRecord>): void
+addCulturalRequirement(req: CulturalRequirement): void
+removeCulturalRequirement(req: CulturalRequirement): void
+setCulturalRequirement(category: String, option: String): void
+getCulturalRequirements(): List<CulturalRequirement>
+setCulturalRequirements(reqs: List<CulturalRequirement>): void
+addSkill(skill: Skill): void
+removeSkill(skill: Skill): void
+getSkills(): List<Skill>
+getSkillsByCategory(category: String): List<Skill>
+setSkills(skills: List<Skill>): void
+toString(): String
-normalizeCensusGender(s: String): String
Skill (Abstract)
+VALID_PROFICIENCY_LEVELS: String[] {static, final}
-dbId: int
-victimSkillDbId: int
-skillName: String
-category: String
-proficiencyLevel: String
+Skill(skillName: String, category: String, proficiencyLevel: String)
+validateProficiencyLevel(level: String): void {static}
+getDbId(): int
+setDbId(dbId: int): void
+getVictimSkillDbId(): int
+setVictimSkillDbId(victimSkillDbId: int): void
+getSkillName(): String
+setSkillName(skillName: String): void
+getCategory(): String
+getProficiencyLevel(): String
+setProficiencyLevel(proficiencyLevel: String): void
+toString(): String
Supply
-dbId: int
-type: String
-quantity: int
-perishable: boolean
-expiryDate: LocalDate
-allocationDate: LocalDate
-description: String
+Supply(type: String, quantity: int)
+Supply(dbId: int, type: String, expiryDate: LocalDate, allocationDate: LocalDate, description: String)
+isExpired(): boolean
+getDbId(): int
+setDbId(dbId: int): void
+getType(): String
+setType(type: String): void
+getQuantity(): int
+setQuantity(quantity: int): void
+isPerishable(): boolean
+setPerishable(perishable: boolean): void
+getExpiryDate(): LocalDate
+setExpiryDate(expiryDate: LocalDate): void
+getAllocationDate(): LocalDate
+setAllocationDate(allocationDate: LocalDate): void
+getDescription(): String
+setDescription(description: String): void
+toString(): String
MockDatabaseQuery (implements DatabaseQueryInterface)
-nextId: int
+MockDatabaseQuery()
-nextId(): int
(all DatabaseQueryInterface methods)
ActionLogger
-instance: ActionLogger {static}
-ACTION_LOG_PATH: Path {static, final}
-ERROR_LOG_PATH: Path {static, final}
-DATE_TIME_FORMATTED: DateTimeFormatter {static, final}
+getInstance(): ActionLogger {static}
+logAdded(entityType: String, entityId: int, description: String): void
+logUpdated(entityType: String, entityId: int, description: String): void
+logSoftDeleted(entityType: String, entityId: int, description: String): void
+logDeleted(entityType: String, entityId: int, description: String): void
+logError(message: String, throwable: Throwable): void
-now(): String
-write(entry: String): void
CulturalRequirement
-dbId: int
-category: String
-option: String
+CulturalRequirement(category: String, option: String)
+getDbId(): int
+setDbId(dbId: int): void
+getCategory(): String
+setCategory(category: String): void
+getOption(): String
+setOption(option: String): void
+toString(): String
RuntimeException
IllegalArgumentException
CulturalOptions <<Serializable>>
~serialVersionUID: long {static, final}
-accommodations: HashMap<String, Set<String>> {final}
+CulturalOptions(accommodations: HashMap<String, Set<String>>)
+getAccommodations(): HashMap<String, Set<String>>
+getOptionsForCategory(category: String): Set<String>
+getCategories(): Set<String>
+isValidOption(category: String, option: String): boolean
FamilyRelation
-dbId: int
-personOne: DisasterVictim
-personTwo: DisasterVictim
-relationshipTo: String
+FamilyRelation(personOne: DisasterVictim, relationshipTo: String, personTwo: DisasterVictim)
+getDbId(): int
+setDbId(dbId: int): void
+getPersonOne(): DisasterVictim
+setPersonOne(personOne: DisasterVictim): void
+getPersonTwo(): DisasterVictim
+setPersonTwo(personTwo: DisasterVictim): void
+getRelationshipTo(): String
+setRelationshipTo(relationshipTo: String): void
IllegalStateException
MedicalRecord
-dbId: int
-location: Location
-treatmentDetails: String
-dateOfTreatment: LocalDate
+MedicalRecord(location: Location, treatmentDetails: String, dateOfTreatment: LocalDate)
+getDbId(): int
+setDbId(dbId: int): void
+getLocation(): Location
+setLocation(location: Location): void
+getTreatmentDetails(): String
+setTreatmentDetails(treatmentDetails: String): void
+getDateOfTreatment(): LocalDate
+setDateOfTreatment(dateOfTreatment: LocalDate): void
<<throws>>